

SKILL SWAP PLATFORM

A PROJECT REPORT

(22ITP62-Project Work)

Submitted by

**BOOMADEVI S
(23ITR016)**

**ARUNPRASAD S
(23ITR008)**

*in partial fulfilment of the
requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

BONAFIDE CERTIFICATE

This is to certify that the Project report entitled “**SKILL SWAP PLATFORM**” is the Bonafide record of project work done by **BOOMADEVI S (23ITR016)** and **ARUNPRASAD S (23ITR008)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in **INFORMATION TECHNOLOGY** of Anna University Chennai during the year 2025 - 2026.

SUPERVISOR

Date:

HEAD OF THE DEPARTMENT

(Signature with seal)

Submitted for the end semester viva voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

DECLARATION

We affirm that the Project Report titled “**SKILL SWAP PLATFORM**” being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

BOOMADEVI S
(23ITR016)

ARUNPRASAD S
(23ITR008)

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date:

Name and Signature of the Supervisor with seal

ABSTRACT

Skill Swap Campus is a full-stack web application designed to facilitate peer-to-peer learning and micro-mentorship within a university environment. The platform enables students to both teach and learn skills by connecting with peers who possess expertise in various domains. Unlike traditional learning systems, Skill Swap Campus promotes a collaborative ecosystem where knowledge exchange happens informally, efficiently, and based on mutual interest.

The system allows students to create skill profiles, offer mentorship, and request learning sessions from other students. Faculty members act as administrators to monitor platform activities, ensuring proper usage and maintaining quality standards. The platform manages scheduling, communication, and tracking of learning sessions, while the actual teaching occurs outside the system through in-person meetings or external tools such as Google Meet or WhatsApp.

Skill Swap Campus incorporates key features such as secure user authentication, skill listing, session management, real-time chat for coordination, and a point-based reward system to encourage active participation. Mentors earn points upon successful completion of sessions, which are reflected in a leaderboard to foster motivation and engagement among students.

Built using the MERN stack (MongoDB, Express.js, React, Node.js), the application ensures scalability, responsiveness, and a modern user experience. Overall, Skill Swap Campus aims to bridge the gap between learners and mentors within academic institutions, fostering a culture of collaborative learning, skill sharing, and continuous personal development.

ACKNOWLEDGEMENT

First and foremost, we acknowledge the abundant grace and presence of Almighty throughout different phases of the project and its successful completion.

We wish to express our gratefulness to our beloved Correspondent **THIRU. E. R. K. KRISHNAN, M.Com.** and all the trust members of Kongu Vellalar Institute of Technology Trust provide all the necessary facilities to complete the project successfully.

We express our deep sense of gratitude to our beloved Principal **Dr. R. PARAMESHWARAN M.E., Ph.D.**, providing us an opportunity to complete the project.

We express our gratitude to **Dr. S. ANANDAMURUGAN M.E., Ph.D.**, Head of the Department, Department of Information Technology for his valuable suggestions.

We are thankful to our Project Coordinators **Dr. E. M. ROOPA DEVI M.E., PH.D.**, **Ms. S. SUJITHA M.Tech.**, and **Ms. G. SASIKALA M.E.**, for their valuable guidance and support to complete our project successfully.

We are thankful to the faculty members of the Department of Information Technology for their valuable guidance and support.

We are grateful to **Mr. Avinash Kumar, Technology Lead at Infosys**, for giving me an opportunity to learn new skills and technologies and enhance my knowledge by providing the necessary infrastructure.

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	iv
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	INTRODUCTION	7
	1.1 OVERVIEW	7
	1.2 OBJECTIVE	7
2	SYSTEM SPECIFICATION	8
	2.1 HARDWARE REQUIREMENTS	8
	2.2 SOFTWARE REQUIREMENTS	8
3	SYSTEM DESIGN	9
	3.1 MODULES DESCRIPTION	9
	3.1.1 User Authentication Module	9
	3.1.2 Student Dashboard Module	9
	3.1.3 Skill Management Module	10
	3.1.4 Session Management Module	10
	3.1.5 Chat Module	11
	3.1.6 Points & Leaderboard Module	12
	3.1.7 Notification Module	12
	3.1.8 Database Module	13
4	RESULTS	14
5	CONCLUSION	15
6	APPENDIX 1-CODING	16
7	REFERENCES	26

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
1.1	SIGNUP PAGE	9
1.2	STUDENT DASHBOARD PAGE	10
1.3	SKILLS MANGEMENT PAGE	10
1.4	SESSION MANAGEMENT PAGE	11
1.5	CHAT PAGE	11
1.6	LEADERBOARD PAGE	12
1.7	NOTIFICATION PAGE	13
1.8	DATABASE SCREEN	13

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CSS	Cascading Style Sheets
C	C Programming Language
HTML	HyperText Markup Language
ID	Identifier
JS	JavaScript
JSX	JavaScript XML
QR	Quick Response (Code)
URL	Uniform Resource Locator
AI	Artificial Intelligence
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
AJAX	Asynchronous JavaScript and XML
I/O	Input / Output
JSON	JavaScript Object Notation
JWT	JSON Web Token

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Skill Swap Campus is a full-stack web application designed to enable peer-to-peer learning and micro-mentorship within a campus environment by allowing students to act as both mentors and learners. The platform allows users to create skill profiles, offer or request learning sessions, and manage scheduling and communication through an integrated system, while the actual teaching takes place outside the platform via in-person meetings or external tools such as Google Meet or WhatsApp. Faculty members serve as administrators to monitor activities, manage users, and ensure proper functioning of the system. The application also includes a point-based reward system where mentors earn points upon successful completion of sessions, contributing to a leaderboard that encourages active participation. Built using the MERN stack (MongoDB, Express.js, React, and Node.js), the platform offers a responsive, user-friendly interface with a modern design, aiming to foster collaboration, knowledge sharing, and continuous skill development among students.

1.2 OBJECTIVE

- To develop a platform that enables peer-to-peer learning among students.
- To allow students to teach and learn skills within the campus community.
- To provide an easy way to create, browse, and manage skill profiles.
- To facilitate session scheduling and coordination between learners and mentors.
- To enable communication through a built-in chat system for session planning.
- To introduce a point-based reward system to motivate student participation.
- To display a leaderboard based on mentor contributions and performance.
- To allow faculty (admins) to monitor users, skills, and sessions effectively.
- To ensure that the platform remains simple, user-friendly, and responsive.
- To promote collaborative learning and knowledge sharing within the campus.

CHAPTER 2

SYSTEM SPECIFICATION

2.1 HARDWARE REQUIREMENTS

Processor	:	Intel Pentium
Processor Speed	:	250MHz to 667MHz
RAM	:	4GB RAM
Hard Disk	:	256GB
Keyboard	:	Standard 104 enhanced
Mouse	:	Local PS/2

2.2 SOFTWARE REQUIREMENTS

Platform	:	Visual Studio Code
Server-Side Script	:	NodeJS
Operating System	:	Windows, macOS, or Linux
Backend	:	MongoDB
Library	:	React Js

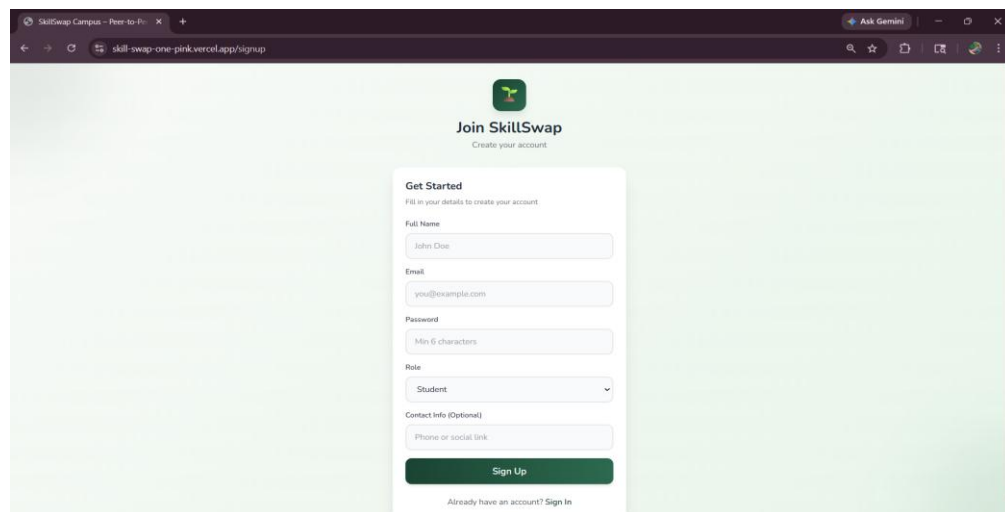
CHAPTER 3

SYSTEM DESIGN

3.1 MODULES DESCRIPTION

3.1.1 User Authentication Module

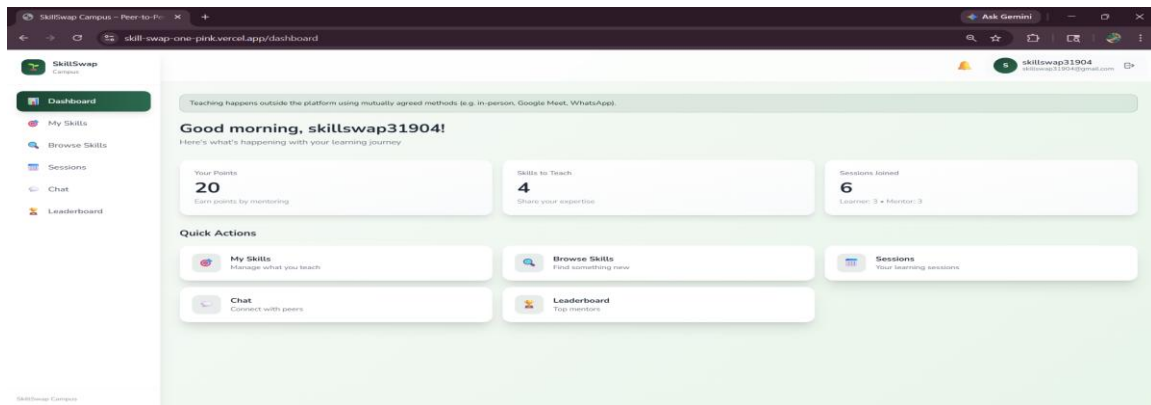
This module provides secure access control for the platform, supporting both Student and Admin roles. It features a robust registration and login system that uses JSON Web Tokens (JWT) for session persistence and Bcrypt for password hashing. To ensure identity security, it integrates an email-based OTP (One-Time Password) verification system for both new account signups and password recovery, restricting platform access to verified campus users only.

A screenshot of a web browser showing the 'Join SkillSwap' sign-up page. The page has a light green background. At the top, there's a logo and the text 'Join SkillSwap' and 'Create your account'. Below this is a 'Get Started' section with the instruction 'Fill in your details to create your account'. The form includes fields for 'Full Name' (with 'John Doe' as a placeholder), 'Email' (with 'you@example.com' as a placeholder), 'Password' (with a 'Min 6 characters' hint), and a 'Role' dropdown menu set to 'Student'. There's also an optional 'Contact Info (Optional)' field for 'Phone or social link'. A green 'Sign Up' button is at the bottom of the form. Below the button, it says 'Already have an account? Sign In'.

1.1 SIGNUP PAGE

3.1.2 Student Dashboard Module

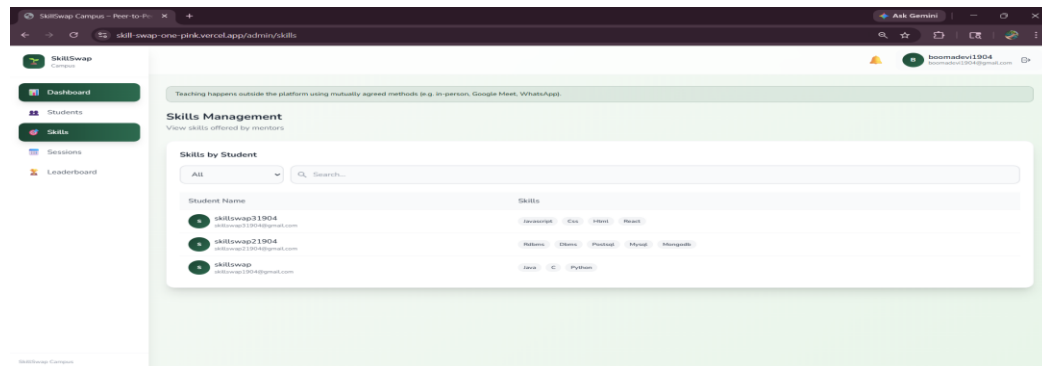
The Student Dashboard serves as the central hub for users to manage their peer-learning activities. It provides a personalized overview of the student's profile, including their earned skill points and quick access to their teaching and learning schedules. From here, students can navigate to their listed skills, track the status of pending session requests, and view recent notifications, creating a streamlined experience for both mentors and learners.



1.2 STUDENT DASHBOARD PAGE

3.1.3 Skill Management Module

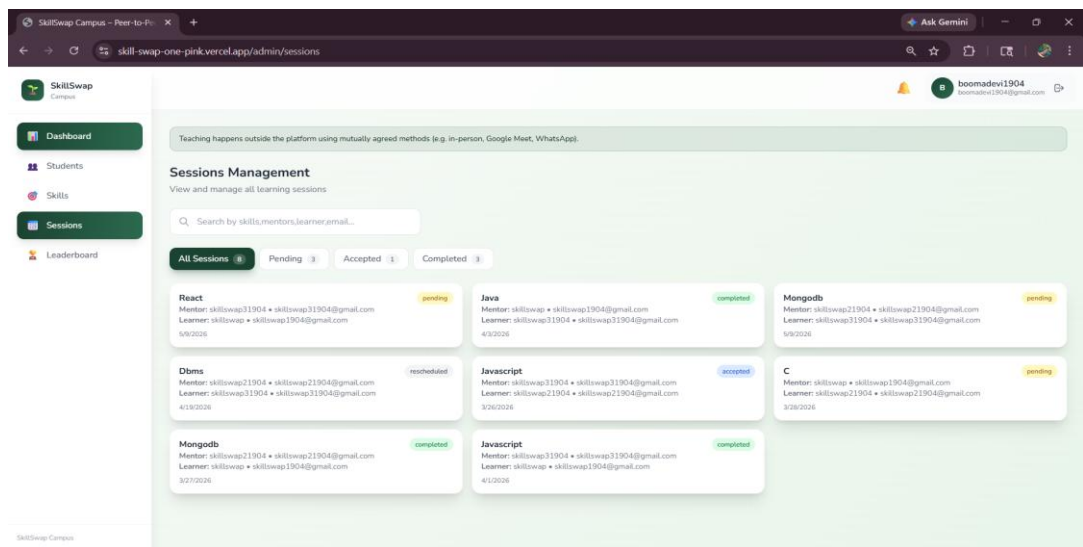
This module allows students to showcase their expertise by listing skills they are willing to teach others. Students can create, update, and delete skill entries, categorizing them (e.g., Technical, Creative, Academic) to make them easily discoverable by others. It also includes a public skill-browsing interface where learners can search for specific competencies and view the profiles of potential mentors before requesting a session.



1.3 SKILL MANAGEMENT PAGE

3.1.4 Session Management Module

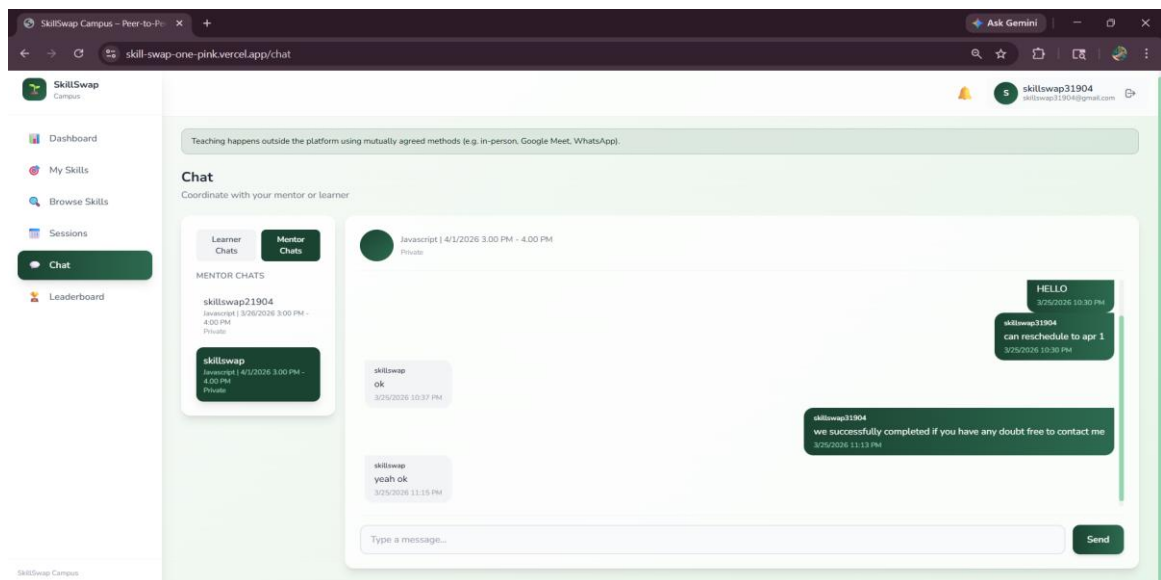
The Session Management module coordinates the end-to-end workflow of a learning engagement between two students. It handles the initial request (including date, time, and teaching mode), allows mentors to accept, reject, or reschedule requests, and manages the transition to a "Completed" status. Once a session is finished, this module triggers the logic to award points to the mentor and opens the feedback loop for both parties.



1.4 SESSION MANAGEMENT PAGE

3.1.5 Chat Module

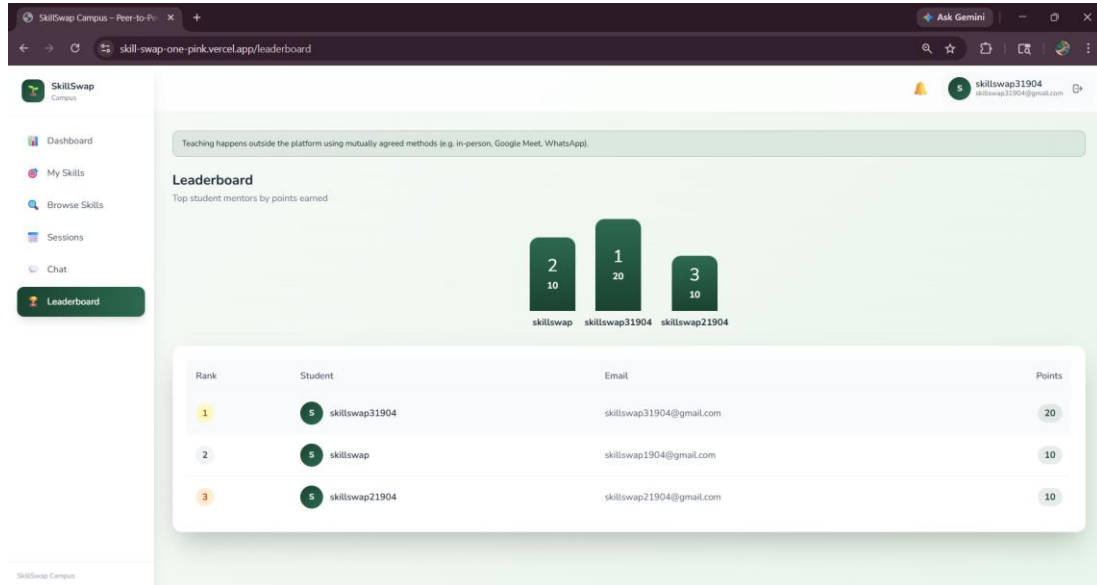
The Chat Module facilitates direct communication between a learner and a mentor once a session request has been accepted. It uses a database-driven messaging system that enables real-time text exchange, allowing users to discuss session details, share external meeting links (like Google Meet or WhatsApp), and coordinate their learning materials. To maintain privacy and focus, chat rooms are strictly tied to specific sessions and are only accessible to the authorized participants.



1.5 CHAT PAGE

3.1.6 Points & Leaderboard Module

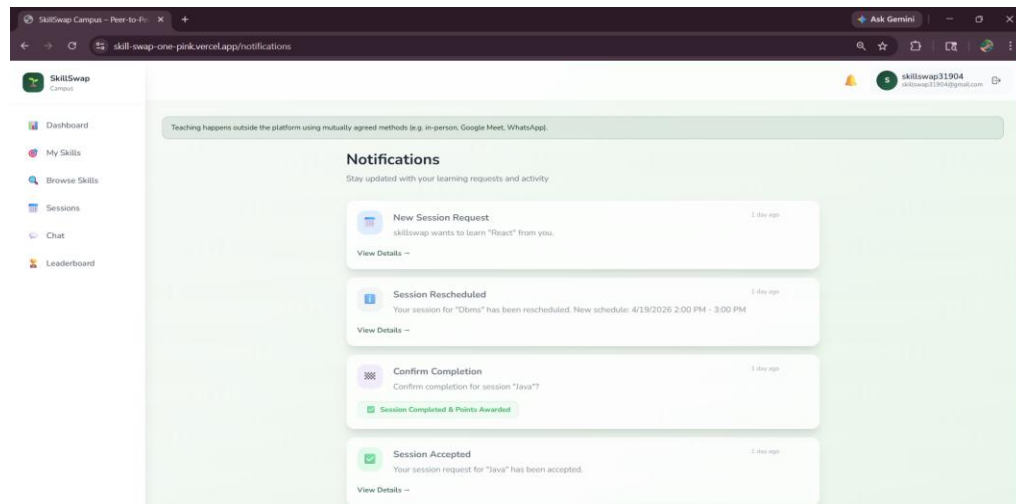
This module gamifies the mentorship experience by tracking "Skill Points" earned by students for their teaching contributions. Every successfully completed session awards a fixed number of points to the mentor, which are then used to rank users on a campus-wide leaderboard. This encourages active participation and recognizes top contributors, fostering a culture of knowledge sharing and healthy academic competition within the university.



1.6 LEADERBOARD PAGE

3.1.7 Notification Module

The Notification Module keeps users informed of all critical platform activities through real-time in-app alerts and dropdown summaries. It triggers automated messages for events such as new session requests, status updates (accepted/rescheduled), and reminders for pending actions like confirming session completion. By centralizing these alerts, the module ensures that students and admins never miss important updates regarding their mentorship commitments.



1.7 NOTIFICATION PAGE

3.1.8 Database Module

The Database Module serves as the persistent storage layer of the application, utilizing MongoDB with Mongoose for structured data modeling. It defines the schemas and relationships for Users, Skills, Sessions, Chats, and Notifications, ensuring data integrity and efficient retrieval. Through its middleware and validation logic, it handles complex operations like cascading updates for point totals and maintaining historical records of all peer-to-peer interactions across the platform.

Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total index size
chatrooms	-	4.10 kB	0 B	0	0 B	1	4.10 kB
chats	-	36.86 kB	1.14 kB	8	142.00 B	1	36.86 kB
notifications	-	36.86 kB	5.49 kB	17	322.00 B	2	73.73 kB
sessions	-	36.86 kB	1.04 kB	6	229.00 B	1	36.86 kB
skills	-	36.86 kB	1.58 kB	12	131.00 B	1	36.86 kB
users	-	36.86 kB	982.00 B	4	245.00 B	2	73.73 kB

1.8 DATABASE SCREEN

CHAPTER 4

RESULT

The Skill Swap Campus project has successfully established a comprehensive peer-to-peer learning platform that enables university students to seamlessly exchange skills within a secure, faculty-monitored ecosystem. By providing dedicated interfaces for both students and administrators, the platform allows users to transition between being learners and mentors through a structured session management workflow. Students can list their expertise in a searchable marketplace, request personalized teaching sessions with defined modes of instruction—such as in-person or online—and coordinate their learning through an integrated, session-specific chat system that ensures privacy and organizational clarity throughout the engagement.

Technically, the project achieves high performance and security by utilizing a decoupled architecture with a React-Vite frontend and a Node.js-Express backend, all backed by a scalable MongoDB database. The implementation of JWT-based authentication and an email-linked OTP verification system ensures that only verified campus members can access the platform, while a gamified Skill Points and leaderboard module incentivizes active mentorship. This results in a robust academic tool that not only fosters community building and knowledge sharing but also provides administrators with the necessary oversight tools to track campus-wide engagement and recognize top student contributors.

CHAPTER 5

CONCLUSION AND FUTURE WORK

In conclusion, the Skill Swap Campus project has successfully addressed the need for a structured, university-centric peer-learning ecosystem by delivering a robust full-stack platform. By integrating secure authentication, a dynamic skill marketplace, and a gamified points system, the application effectively transforms the campus into a collaborative community where students are empowered to both teach and learn. The platform not only streamlines the logistical challenges of scheduling and communication through its automated session workflows and integrated chat but also provides faculty with the oversight tools necessary to maintain a productive academic environment, ultimately fostering a culture of voluntary knowledge exchange and professional growth.

For future work, the platform is poised for several strategic enhancements to further deepen student engagement and scalability. Key areas for development include the integration of real-time WebSocket's for instantaneous messaging, the implementation of a mobile application using React Native for on-the-go access, and an AI-driven recommendation engine to suggest skills and mentors based on a student's interests and learning history. Additionally, expanding the platform to support group workshops and integrating a video conferencing API like Jitsi or Zoom directly into the interface would provide a more comprehensive, all-in-one virtual learning experience that transcends physical campus boundaries.

APPENDIX 1

CODING

App.jsx :

```
import React from "react";
import { Routes, Route, Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import Layout from "../components/Layout";
import Login from "../pages/Login";
import Signup from "../pages/Signup";
import VerifyOTP from "../pages/VerifyOTP";
import ForgotPassword from "../pages/ForgotPassword";
import ResetPassword from "../pages/ResetPassword";
import StudentDashboard from "../pages/StudentDashboard";
import SkillProfile from "../pages/SkillProfile";
import SkillListing from "../pages/SkillListing";
import SessionRequests from "../pages/SessionRequests";
import Chat from "../pages/Chat";
import AdminDashboard from "../pages/AdminDashboard";
import Leaderboard from "../pages/Leaderboard";
import Notifications from "../pages/Notifications";
import { Toaster } from "react-hot-toast";

function HomeRedirect() {
  const { user } = useAuth();
  return <Navigate to={user?.role === "admin" ? "/admin" : "/dashboard"} replace />;
}

function PrivateRoute({ children, studentOnly, adminOnly }) {
  const { user, loading } = useAuth();
  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-surface">
        <div className="text-primary font-medium">Loading...</div>
      </div>
    );
  }
  if (!user) return <Navigate to="/login" replace />;
  if (studentOnly && user.role !== "student") return <Navigate to="/admin" replace />;
  if (adminOnly && user.role !== "admin") return <Navigate to="/dashboard" replace />;
  return children;
}
```

```

export default function App() {
  return (
    <>
    <Toaster position="top-right" />
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/signup" element={<Signup />} />
      <Route path="/verify-otp" element={<VerifyOTP />} />
      <Route path="/forgot-password" element={<ForgotPassword />} />
      <Route path="/reset-password" element={<ResetPassword />} />
      <Route
        path="/"
        element={
          <PrivateRoute>
            <Layout />
          </PrivateRoute>
        }
      />
      <Route index element={<HomeRedirect />} />
      <Route path="dashboard" element={<PrivateRoute studentOnly><StudentDashboard
/></PrivateRoute>} />
      <Route path="skills" element={<PrivateRoute studentOnly><SkillProfile /></PrivateRoute>} />
      <Route path="browse" element={<PrivateRoute studentOnly><SkillListing /></PrivateRoute>} />
      <Route path="sessions" element={<PrivateRoute studentOnly><SessionRequests
/></PrivateRoute>} />
      <Route path="chat" element={<PrivateRoute studentOnly><Chat /></PrivateRoute>} />
      <Route path="chat/:sessionId" element={<PrivateRoute studentOnly><Chat /></PrivateRoute>}
/>
      <Route path="leaderboard" element={<PrivateRoute studentOnly><Leaderboard
/></PrivateRoute>} />
      <Route path="notifications" element={<PrivateRoute studentOnly><Notifications
/></PrivateRoute>} />
      <Route path="admin" element={<PrivateRoute adminOnly><AdminDashboard
/></PrivateRoute>} />
      <Route path="admin/students" element={<PrivateRoute adminOnly><AdminDashboard
tab="students" /></PrivateRoute>} />
      <Route path="admin/skills" element={<PrivateRoute adminOnly><AdminDashboard tab="skills"
/></PrivateRoute>} />
      <Route path="admin/sessions" element={<PrivateRoute adminOnly><AdminDashboard
tab="sessions" /></PrivateRoute>} />
      <Route path="admin/leaderboard" element={<PrivateRoute adminOnly><AdminDashboard
tab="leaderboard" /></PrivateRoute>} />
    </Route>
    <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  </>
);
}

```

StudentDashboard.jsx:

```
import React, { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import { api } from "../api";

export default function StudentDashboard() {
  const { user, updateUser } = useAuth();
  const [mySkillsCount, setMySkillsCount] = useState(0);
  const [sessionsLearnerCount, setSessionsLearnerCount] = useState(0);
  const [sessionsMentorCount, setSessionsMentorCount] = useState(0);

  useEffect(() => {
    api.get("/api/auth/me").then((r) => updateUser(r.data)).catch(() => {});
  }, [updateUser]);

  useEffect(() => {
    if (!user) return;
    api
      .get("/api/skills/my")
      .then((r) => setMySkillsCount((r.data || []).length))
      .catch(() => setMySkillsCount(0));
    api
      .get("/api/sessions/my")
      .then((r) => {
        const asLearner = ((r.data && r.data.asLearner) || []);
        const asMentor = ((r.data && r.data.asMentor) || []);
        setSessionsLearnerCount(asLearner.length);
        setSessionsMentorCount(asMentor.length);
      })
      .catch(() => {
        setSessionsLearnerCount(0);
        setSessionsMentorCount(0);
      });
  }, [user]);

  const getGreeting = () => {
    const hour = new Date().getHours();
    if (hour < 12) return "Good morning";
    if (hour < 17) return "Good afternoon";
    return "Good evening";
  };
}
```

```

const stats = [
  { label: "Your Points", value: user?.points ?? 0, description: "Earn points by mentoring"
},
  { label: "Skills to Teach", value: mySkillsCount || 0, description: "Share your expertise"
},
  { label: "Sessions Joined", value: (sessionsLearnerCount + sessionsMentorCount) || 0,
description: `Learner: ${sessionsLearnerCount} • Mentor: ${sessionsMentorCount}` },
];

const quickLinks = [
  { to: "/skills", label: "My Skills", desc: "Manage what you teach", icon: "🎯" },
  { to: "/browse", label: "Browse Skills", desc: "Find something new", icon: "🔍" },
  { to: "/sessions", label: "Sessions", desc: "Your learning sessions", icon: "📅" },
  { to: "/chat", label: "Chat", desc: "Connect with peers", icon: "💬" },
  { to: "/leaderboard", label: "Leaderboard", desc: "Top mentors", icon: "🏆" },
];

return (
  <div className="animate-fade-in">
    { /* Welcome section */ }
    <div className="mb-8">
      <h1 className="text-3xl font-bold text-gray-800">
        {getGreeting()}, {user?.name?.split(" ")[0]}!
      </h1>
      <p className="text-gray-500 mt-1">Here's what's happening with your learning
journey</p>
    </div>

    { /* Stats grid */ }
    <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3 mb-8">
      {stats.map((stat, index) => (
        <div
          key={stat.label}
          className="stat-card animate-fade-in"
          style={{ animationDelay: `${index * 100}ms` }}
        >
          <div className="flex items-start justify-between">
            <div>
              <p className="text-gray-500 text-sm font-medium">{stat.label}</p>
              <p className="text-4xl font-bold text-gray-800 mt-2">{stat.value}</p>
              <p className="text-gray-400 text-sm mt-1">{stat.description}</p>
            </div>
          </div>
        </div>
      ))}
    </div>
  </div>
)

```

```

    { /* Quick links */ }
    <h2 className="text-xl font-bold text-gray-800 mb-4">Quick Actions</h2>
    <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
      {quickLinks.map((link, index) => (
        <Link
          key={link.to}
          to={link.to}
          className={`card-elevated p-5 group animate-fade-in flex items-center gap-4`}
          style={{ animationDelay: `${(index + 3) * 100}ms` }}
        >
          <div className="w-12 h-12 rounded-xl bg-primary/10 flex items-center justify-
center text-xl transition-transform group-hover:scale-110">
            {link.icon}
          </div>
          <div>
            <h3 className="font-bold text-gray-800 group-hover:text-primary transition-
colors">
              {link.label}
            </h3>
            <p className="text-gray-500 text-sm">{link.desc}</p>
          </div>
        </Link>
      ))}
    </div>
  </div>
);
}

```

Login.jsx :

```

import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import { api } from "../api";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);

```

```

try {
  const res = await api.post("/api/auth/login", { email, password });
  const data = res.data;
  login(data.user, data.token);
  navigate(data.user.role === "admin" ? "/admin" : "/dashboard");
} catch (err) {
  setError(err.response?.data?.error || err.message || "Login failed");
  setLoading(false);
}
};

return (
  <div className="min-h-screen flex items-center justify-center p-4">
    { /* Background decoration */ }
    <div className="absolute inset-0 overflow-hidden pointer-events-none">
      <div className="absolute -top-40 -right-40 w-80 h-80 bg-primary/10 rounded-full blur-3xl" />
      <div className="absolute -bottom-40 -left-40 w-80 h-80 bg-primary-light/10 rounded-full blur-3xl" />
    </div>

    <div className="relative w-full max-w-md">
      { /* Logo */ }
      <div className="text-center mb-8">
        <div className="inline-flex items-center justify-center w-16 h-16 rounded-2xl bg-gradient-to-br from-primary to-primary-light text-white text-3xl mb-4 shadow-lg">

        </div>
        <h1 className="text-3xl font-bold text-gray-800">SkillSwap Campus</h1>
        <p className="text-gray-500 mt-1">Peer-to-Peer Learning</p>
      </div>

      <div className="card-elevated">
        <h2 className="text-xl font-bold text-gray-800 mb-2">Welcome back!</h2>
        <p className="text-gray-500 text-sm mb-6">Enter your details to access your account</p>

        {error && (
          <div className="bg-red-50 border border-red-200 text-red-700 px-4 py-3 rounded-xl text-sm flex items-center gap-2 mb-4">
            <span> ⚠ </span>
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-semibold text-gray-700 mb-2">Email</label>
            <input
              type="email"

```

```

        value={email}
        onChange={(e) => setEmail(e.target.value)}
        className="input-field"
        placeholder="you@example.com"
        required
      />
    </div>

    <div>
      <label className="block text-sm font-semibold text-gray-700 mb-2">Password</label>
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        className="input-field"
        placeholder="••••••••"
        required
      />
    </div>

    <div className="flex justify-end mt-1">
      <Link to="/forgot-password" size="sm" className="text-sm text-primary hover:underline">
        Forgot password?
      </Link>
    </div>

    <button
      type="submit"
      className="btn-primary w-full py-3 text-lg mt-4"
      disabled={loading}
    >
      {loading ? "Signing in..." : "Sign In"}
    </button>
  </form>

  <p className="mt-6 text-center text-gray-600">
    Don't have an account?{" "}
    <Link to="/signup" className="text-primary font-semibold hover:underline">
      Create one
    </Link>
  </p>
</div>
</div>
);
}

```

Signup.jsx:

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { api } from "../api";

export default function Signup() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [role, setRole] = useState("student");
  const [contact, setContact] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await api.post("/api/auth/signup", {
        name,
        email,
        password,
        role,
        contact: contact || undefined,
      });
      // Instead of logging in, redirect to OTP verification
      navigate("/verify-otp", { state: { email } });
    } catch (err) {
      const msg = err.response?.data?.error || err.response?.data?.errors?.[0]?.msg || err.message;
      setError(msg || "Signup failed");
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center p-4 py-8">
      { /* Background decoration */ }
      <div className="absolute inset-0 overflow-hidden pointer-events-none">
        <div className="absolute -top-40 -left-40 w-80 h-80 bg-primary/10 rounded-full blur-3xl" />
        <div className="absolute -bottom-40 -right-40 w-80 h-80 bg-primary-light/10 rounded-full blur-3xl" />
      </div>

      <div className="relative w-full max-w-md">
        { /* Logo */ }
        <div className="text-center mb-8">
          <div className="inline-flex items-center justify-center w-16 h-16 rounded-2xl bg-
```

gradient-to-br from-primary to-primary-light text-white text-3xl mb-4 shadow-lg">



```
</div>
<h1 className="text-3xl font-bold text-gray-800">Join SkillSwap</h1>
<p className="text-gray-500 mt-1">Create your account</p>
</div>
```

```
<div className="card-elevated">
  <h2 className="text-xl font-bold text-gray-800 mb-2">Get Started</h2>
  <p className="text-gray-500 text-sm mb-6">Fill in your details to create your
account</p>
```

```
  {error && (
    <div className="bg-red-50 border border-red-200 text-red-700 px-4 py-3 rounded-xl
text-sm flex items-center gap-2 mb-4">
      <span> ⚠️ </span>
      {error}
    </div>
  )}
```

```
<form onSubmit={handleSubmit} className="space-y-4">
  <div>
    <label className="block text-sm font-semibold text-gray-700 mb-2">Full Name</label>
    <input
      type="text"
      value={name}
      onChange={(e) => setName(e.target.value)}
      className="input-field"
      placeholder="John Doe"
      required
    />
  </div>
```

```
  <div>
    <label className="block text-sm font-semibold text-gray-700 mb-2">Email</label>
    <input
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      className="input-field"
      placeholder="you@example.com"
      required
    />
  </div>
```

```
  <div>
    <label className="block text-sm font-semibold text-gray-700 mb-2">Password</label>
    <input
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
    />
  </div>
```

```

        className="input-field"
        placeholder="Min 6 characters"
        minLength={6}
        required
      />
    </div>

    <div>
      <label className="block text-sm font-semibold text-gray-700 mb-2">Role</label>
      <select
        value={role}
        onChange={(e) => setRole(e.target.value)}
        className="input-field"
      >
        <option value="student">Student</option>
        <option value="admin">Admin</option>
      </select>
    </div>

    <div>
      <label className="block text-sm font-semibold text-gray-700 mb-2">Contact Info
(Optional)</label>
      <input
        type="text"
        value={contact}
        onChange={(e) => setContact(e.target.value)}
        className="input-field"
        placeholder="Phone or social link"
      />
    </div>

    <button
      type="submit"
      className="btn-primary w-full py-3 text-lg mt-2"
      disabled={loading}
    >
      {loading ? "Sending OTP..." : "Sign Up"}
    </button>
  </form>

  <p className="mt-6 text-center text-gray-600">
    Already have an account?{" "}
    <Link to="/login" className="text-primary font-semibold hover:underline">
      Sign In
    </Link>
  </p>
</div>
</div>
</div>
);
}

```

REFERENCES

1. **React js Documentation**-React learn,<https://react.dev/learn>
2. **Node.js Documentation** – OpenJS Foundation, <https://nodejs.org/en/docs>
3. **Express.js Guide** – Express.js Official Site, <https://expressjs.com/>
4. **MongoDB Manual** – MongoDB Inc., <https://www.mongodb.com/docs/manual/>
5. **Genai** - IBM Official Site,<https://www.ibm.com/think/topics/generative-ai>
6. **Host Link** - <https://skill-swap-one-pink.vercel.app/>
7. **GIT HUB** : <https://github.com/boomadevi-1904/Skill-Swap.git>